PHP Orientado a Objetos com Banco de Dados

Guia roteirizado para construir a aplicação do zero até a versão do professor



Classes • Constructor • Métodos • Encapsulamento • Herança • Polimorfismo • Abstração • PDO

O que vamos construir

Uma aplicação navegável, funcional, orientada a objetos e conectada ao banco.

1. Banco

Importar a base demo e interpretar o EER antes de programar.

2. Classes

Criar entidades baseadas nas tabelas: Produto, Cliente, Funcionario, Producao.

3. CRUD

Listar, cadastrar, editar, visualizar e excluir dados com métodos.

4. Repositórios

Separar SQL da tela usando Repository e Database.

5. Comparação

Ver PDO/OOP e MySQLi procedural para comparar manutenção.

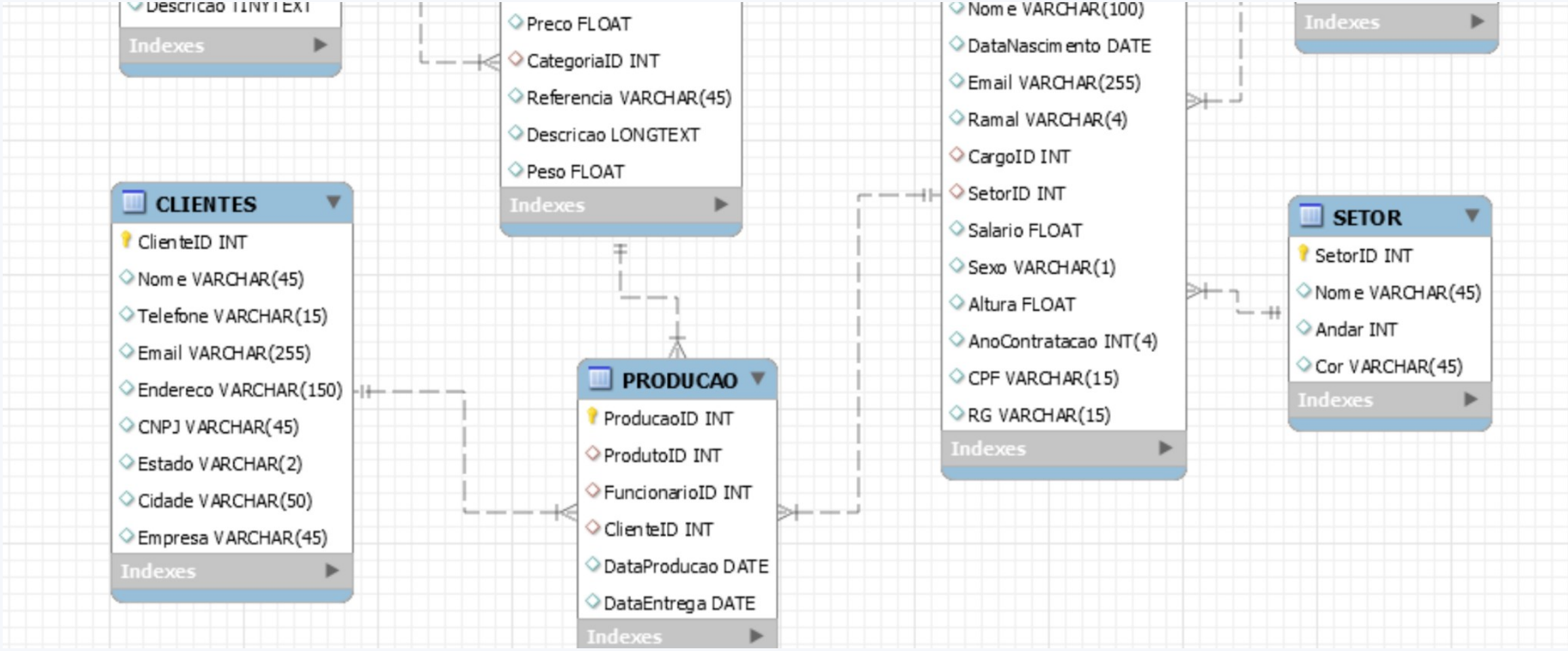
6. Versão aluno

Base incompleta para o aluno construir por etapas.

Resultado: o aluno entende como o modelo relacional vira objetos e telas reais.

Modelo EER como ponto de partida

O código nasce da leitura das entidades, chaves primárias e relacionamentos.



Entidades principais: cargos, setor, funcionarios, categorias, produtos, clientes e producao.

Da tabela para a classe

Cada tabela inspira atributos; cada regra útil vira método.

```
class Produto extends AbstractEntity {
  public static function tableName(): string {
    return 'produtos';
  }
  public function precoFormatado(): string {
    return 'R$ ' . number_format((float)$this-
>get('Preco'),2,',','.');
  }
  public function calcularFreteEstimado(float
$valorPorKg): float {
    return (float)$this->get('Peso',0) *
$valorPorKg;
  }
}
```

Atributos

Nome, Preco, CategoriaID, Referencia, Descricao e Peso.

Métodos

precoFormatado() e calcularFreteEstimado(float $valorPorKg).

Objeto

Um registro do banco vira uma instância de Produto em memória.

Ideia central: tabela guarda dados; classe organiza dados e comportamento.

Encapsulamento e construtor

Os dados entram no objeto por uma porta controlada.

```
abstract class AbstractEntity {
  protected array $dados = [];

  public function __construct(array $dados = []) {
    $this->fill($dados);
  }

  public function get(string $atributo, mixed
$padrao = null): mixed {
    return $this->dados[$atributo] ?? $padrao;
  }

  public function set(string $atributo, mixed
$valor): void {
    $this->dados[$atributo] = $valor;
  }
}
```

Constructor

Recebe um array do banco ou do formulário e hidrata o objeto.

protected $dados

Evita acesso descontrolado ao estado interno do objeto.

get() e set()

Criam uma interface mínima para leitura e alteração.

Encapsular não é esconder por esconder; é controlar como o estado é acessado e modificado.

Herança: Pessoa, Cliente e Funcionário

Classes especializadas reaproveitam comportamento comum.

```
abstract class Pessoa extends AbstractEntity {
  public function getNome(): string {
    return (string)$this->get('Nome','Sem
nome');
  }
  public function getEmail(): string {
    return (string)$this->get('Email','');
  }
}

class Cliente extends Pessoa {}
class Funcionario extends Pessoa {}
```

Por que usar herança?

Cliente e Funcionário compartilham nome e email, mas têm finalidades diferentes.

Quando evitar?

Não use herança se a relação não for claramente “é um tipo de”.

Na aula

Peça ao aluno para identificar quais entidades podem herdar de Pessoa.

Herança deve reduzir repetição sem tornar a hierarquia artificial.

Polimorfismo: o mesmo método, respostas diferentes

A aplicação chama resumo(), mas cada objeto decide como se apresentar.

```
$objetos = [
  new Produto(['Nome'=>'Chave','Preco'=>12.50]),
  new Cliente(['Nome'=>'Cliente Escola']),
  new Funcionario(['Nome'=>'Professor
PHP','Salario'=>4500]),
  new Producao(['ProducaoID'=>101])
];

foreach ($objetos as $obj) {
  echo $obj->resumo();
}
```

Produto

Produto: Chave - R$ 12,50

Cliente

Cliente: Cliente Escola | empresa...

Funcionário

Funcionário: Professor PHP | salário anual...

Polimorfismo simplifica a tela: ela não precisa conhecer todos os detalhes de cada classe.

Abstração: Database e Repository

A tela não deve carregar SQL espalhado.

Database

Centraliza conexão, charset, tratamento de erro e credenciais.

Repository

Implementa all(), find(), create(), update() e delete().

Telas

Chamam métodos do Repository e exibem os dados.

```
$repo = new ProdutoRepository(Database::getInstance()->getConnection());
$produtos = $repo->all();
$produto = $repo->find(10);
$novoId = $repo->create($_POST);
```

Separar responsabilidades melhora leitura, manutenção, testes e reaproveitamento.

Fluxo da navegação CRUD

A aplicação final do professor fica completa e navegável.

index.php

Dashboard com contadores e exemplos OOP.

list.php

Tabela com registros do módulo selecionado.

form.php

Formulário de inclusão ou edição.

save.php

Recebe POST e chama create/update.

delete.php

Remove registro e volta para a lista.

```
list.php?entity=produtos
form.php?entity=funcionarios&id=12
view.php?entity=clientes&id=3
delete.php?entity=producao&id=21
```

A variável entity permite reaproveitar a mesma estrutura para produtos, clientes, funcionários e demais módulos.

Duas versões finais para comparação

O aluno enxerga a diferença entre organização OOP/PDO e acesso procedural.

Versão Professor PDO

Mais alinhada com OOP: Database singleton, entidades, GenericRepository, prepared statements e telas reaproveitáveis.

Use como versão principal e gabarito técnico.

Versão MySQLi procedural

Mesmo banco e mesma navegação, mas com funções de acesso. Serve para discutir acoplamento, repetição, segurança e manutenção.

Use para comparação didática.

A comparação ajuda o aluno a perceber por que arquitetura importa em aplicações reais.

Roteiro do aluno: do zero até o professor

Use a pasta aluno_base como ponto de partida.

Etapa 1

Importar o SQL e validar as tabelas no MySQL.

Etapa 2

Completar Database.php com PDO.

Etapa 3

Criar Produto e ProdutoRepository.

Etapa 4

Criar listagem de produtos.

Etapa 5

Criar formulário e save.php.

Etapa 6

Replicar para clientes e funcionários.

Fechamento

Comparar o próprio código com professor_pdo e justificar onde aparecem encapsulamento, herança, polimorfismo e abstração.

A versão do professor não deve ser aberta no início: ela funciona como referência ao final da construção.

Checklist final de entrega

Use estes critérios para avaliar a atividade.

Banco

SQL importado, tabelas com dados e EER compreendido.

CRUD

Navegação completa por dashboard, lista, formulário, salvar, editar e excluir.

OOP

Classes, construtores, métodos, parâmetros, atributos e encapsulamento implementados.

Conceitos avançados

Herança, polimorfismo e abstração demonstrados no código.

Arquitetura

Database e Repository isolando acesso ao banco.

Comparação

Aluno explica diferenças entre PDO/OOP e MySQLi procedural.

Produto final: aplicação navegável + material explicativo + roteiro de construção.

Ao final, o aluno deve conseguir defender as escolhas técnicas e apontar onde cada conceito aparece no projeto.